

TECHNICAL REPORT

FruGo

Full-Stack Fresh Produce E-Commerce Platform with OTP Auth & Admin Approval Workflow

AUTHOR

Vasant Konar

CONTACT

vasant@crew.inc

DATE

June 2026

VERSION

1.0 — Initial Release

REPOSITORY

github.com/vasant24konar/FruGoHP 8.2 · Laravel 11 · MySQL

STACK

8 · Docker

TABLE OF CONTENTS

1. Executive Summary
2. Introduction & Scope
3. System Architecture
4. Technology Stack
5. Database Design
6. Module Descriptions
7. Authentication & RBAC
8. OTP Login Flow
9. Security Design
10. Deployment & Infrastructure
11. Challenges & Solutions
12. Testing Strategy

13. Future Enhancements

14. Conclusion

SECTION 01

Executive Summary

FruGo is a full-stack e-commerce web application built on **Laravel 11 (PHP 8.2)** for selling fresh fruits and vegetables online — a complete platform combining a customer-facing shop with a back-office administration system in a single application.

Three roles — Administrator, Product Manager, and Customer — each have distinct capabilities. Customers browse an approved product catalogue, add items to a session-based cart, and place orders, all authenticated via **passwordless OTP login** (a 6-digit code emailed to them). Admins and Product Managers use traditional email/password login and operate the back-office: creating products, managing approval workflows, and administering user accounts.

The application is containerised with Docker Compose, deployable with a single command, and structured around SOLID principles using Repository and Service design patterns.

SECTION 02

Introduction & Scope

Purpose

FruGo delivers a frictionless shopping experience for customers (no password to remember) while maintaining controlled product publishing — products created by Product Managers are only visible in the shop after explicit approval by an Administrator.

In Scope

- **Customer shop** — product catalogue, search, category filter, product detail pages

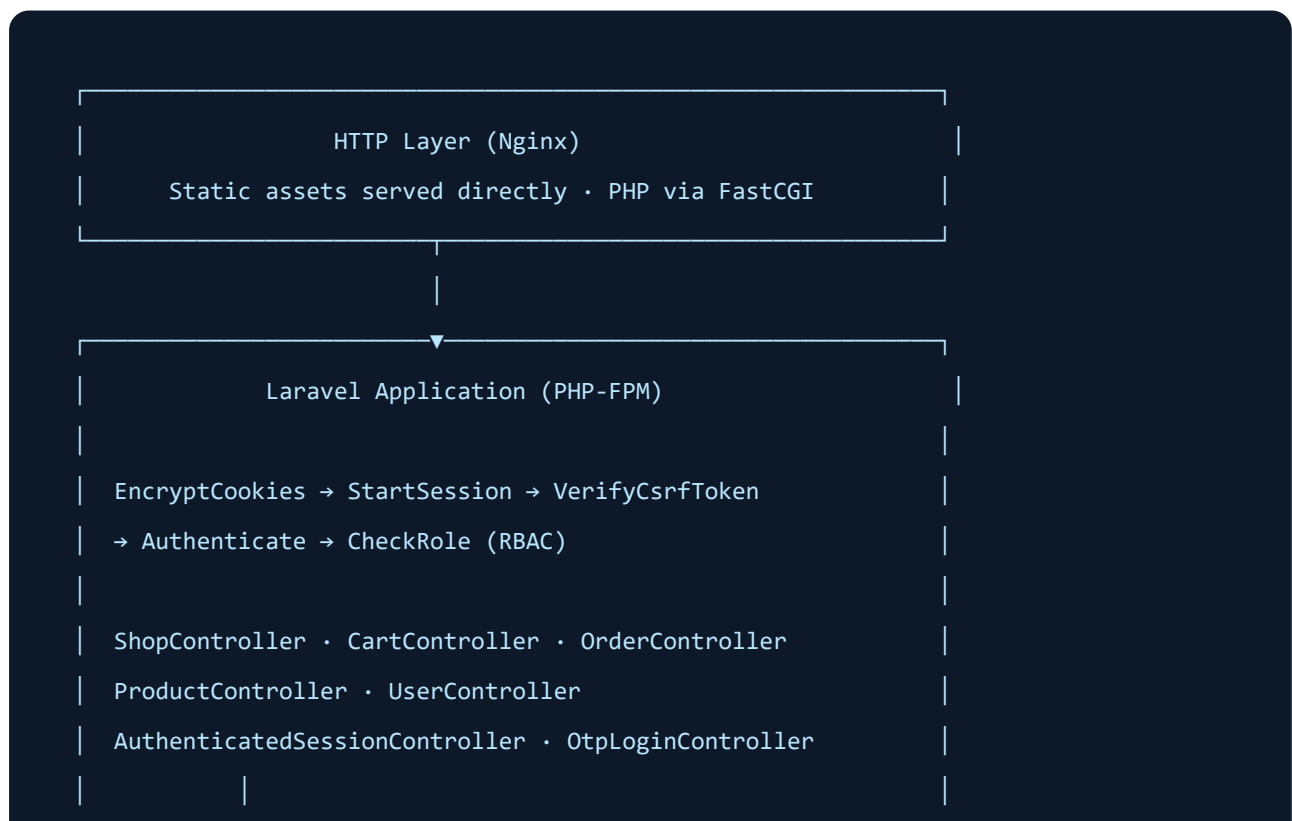
- **Session cart** — add, update quantity, remove, clear; persisted in PHP session
- **Checkout & order tracking** — place order from cart, view order history and status
- **OTP passwordless auth** — customers authenticate via 6-digit email code (10-minute expiry)
- **Admin / PM back-office** — full CRUD product management, approval workflow
- **Admin user management** — create staff accounts, assign roles
- **Email notifications** — OTP delivery, product approved/rejected status emails
- **Docker Compose** four-service containerised local environment

Out of Scope

- Payment gateway integration
- Real-time notifications or WebSockets
- Product image file upload (managed via image path references)

SECTION 03

System Architecture





Design Patterns

PATTERN	WHERE USED	BENEFIT
Repository	ProductRepository / ProductRepositoryInterface	Decouples query logic from business rules; independently testable
Service Layer	ProductService	Single place for approval rules and ownership checks
Dependency Injection	RepositoryServiceProvider	Controllers depend on contracts, not concrete classes
Middleware Pipeline	CheckRole, Authenticate	Cross-cutting access control without code duplication
Session Cart	CartController + PHP session	Zero-schema cart; automatically expires with session

SECTION 04

Technology Stack

Component	Technology	Version	Justification
Language	PHP	8.2	Latest stable; strict_types, named arguments, readonly properties
Framework	Laravel	11	Built-in Auth, Eloquent, Blade, Mail, Session — all features needed without extra packages
Database	MySQL	8.0	FULLTEXT indexes, strong Eloquent driver, wide hosting support
Web Server	Nginx	1.25-alpine	Serves CSS/JS directly; FastCGI pass-through to PHP-FPM
Containerisation	Docker + Compose	v2	Reproducible environment across developer machines
Dev Email	MailHog	latest	Catches all OTP emails and notifications locally — no real SMTP needed in dev
Frontend	Bootstrap 5 + Blade	5.3	Server-rendered; no JS build step; responsive out of the box
Code Style	Laravel Pint	1.x	PSR-12 enforced automatically

SECTION 05

Database Design

```

users (id, name, email, password, role, timestamps)
|
|—< products (id, title, description, price, stock, category,
|           image_path, status, rejection_reason, date_available,
|           created_by → users.id, approved_by → users.id,
|           approved_at, deleted_at, timestamps)
|

```

```
└─< orders (id, user_id → users.id, status, total,  
|         shipping_address, notes, timestamps)  
|         └─< order_items (order_id, product_id, quantity,  
|                           unit_price, timestamps)  
|  
└─< otp_tokens (id, email, token [bcrypt hash], expires_at, timestamps)  
  
users.role      ENUM: admin | product_manager | customer  
products.status ENUM: pending | approved | rejected  
orders.status   ENUM: pending | processing | shipped | delivered | cancelled
```

TABLE	PURPOSE	KEY DESIGN DECISION
users	All users across all roles	Single table with role discriminator — no joins needed for auth checks
products	Product catalogue with approval workflow	Soft-deletes preserve history; status enum drives the workflow state machine
orders	Customer orders placed at checkout	shipping_address snapshotted at order time
order_items	Line items per order	unit_price snapshotted — price changes never retroactively affect past orders
otp_tokens	One-time login codes for customers	Stored as bcrypt hash; deleted immediately after successful verification

SECTION 06

Module Descriptions

ShopController

Public product listing and detail. `index()` returns paginated approved products with search and category filter. No authentication required — the shop is publicly browsable.

CartController

Session-based cart stored as `session('cart') = [productId => ['qty' => N, 'price' => X]]`. Methods: add, update, remove, clear, index. Requires `role:customer` middleware — admins and managers cannot use the cart.

OrderController

`store()` runs in a database transaction: creates the `Order`, writes all `OrderItem` rows with snapshotted prices, then clears the session cart atomically. `index()` and `show()` serve order history and detail to the authenticated customer.

OtpLoginController

Full passwordless OTP flow — generates a 6-digit code, bcrypt-hashes it before storage, sends via email, verifies with `Hash::check()`, and deletes the token on success. See Section 8 for the complete flow.

ProductController

RESTful product management for admin and product_manager roles. Delegates business logic to `ProductService`. Additional actions beyond CRUD: `approve()`, `reject()`, `submit()`. Sends email notifications on approval and rejection.

Admin\UserController

Admin-only staff account management. Customer accounts are auto-created by `OtpLoginController` on first login — this controller only creates and manages admin and product_manager accounts.

SECTION 07

Authentication & RBAC

METHOD	USED BY	ROUTE
Email + Password	Admin, Product Manager	POST /login

METHOD	USED BY	ROUTE
OTP Passwordless	Customers	POST /login/otp → POST /login/otp/verify

Role Permission Matrix

FEATURE	ADMIN	PRODUCT_MANAGER	CUSTOMER
Browse shop	✓	✓	✓ (public)
Cart / checkout / orders	X	X	✓
Create product	✓ auto-approved	✓ goes pending	X
Edit / delete product	✓ any	✓ own only	X
Approve / Reject product	✓	X	X
Dashboard + order overview	✓	✓	X
Manage user accounts	✓	X	X

SECTION 08

OTP Login Flow

Step 1 – Request OTP

```

Customer enters email → POST /login/otp
→ Find or auto-create User with role=customer
→ Generate random 6-digit code
→ bcrypt hash → store in otp_tokens(email, hash, expires_at=now+10min)
→ Mail::to($email)->send(new OtpMail($code))
→ Redirect to verify form

```

Step 2 – Verify OTP

```

Customer enters code → POST /login/otp/verify

```



```
→ Lookup token by email
→ Check expires_at > now()
→ Hash::check($submitted, $storedHash)
→ DELETE token immediately ← prevents replay attacks
→ Auth::login($user) → session()->regenerate()
→ Redirect to shop homepage
```

Resend

```
POST /login/otp/resend → delete existing token → restart Step 1
```

SECURITY PROPERTY	HOW ACHIEVED
Token secrecy at rest	bcrypt hash stored — DB breach does not expose valid codes
Short lifetime	10-minute server-side expiry enforced on every verify attempt
Single use	Token deleted immediately on successful verification
No enumeration	Same error message for wrong, expired, or missing token
Session fixation prevention	session()->regenerate() called immediately after Auth::login()
Auto account creation	New customer accounts created on first OTP request — no registration form needed

DEVELOPMENT MODE

When APP_ENV=local the OTP code is shown on the verify page and captured by MailHog at <http://localhost:8025>. In production only the email delivery path is used.

SECTION 09

Security Design

THREAT	CONTROL	IMPLEMENTATION
SQL Injection	Parameterised queries	All queries via Eloquent ORM — PDO prepared statements throughout
Stored XSS	Sanitisation + output encoding	strip_tags() allowlist in ProductRequest; Blade {{ }} encodes all output
CSRF	Synchroniser token	@csrf on all state-changing forms; VerifyCsrfToken middleware
OTP Brute Force	Short expiry + single use	10-minute window; token deleted on first correct use
Password Brute Force	Rate limiting	5 attempts per email/IP per minute → HTTP 429
Session Fixation	Session regeneration	session()->regenerate() on both login methods
Credential Storage	bcrypt	Passwords and OTP tokens both hashed — same security model
Privilege Escalation	Multi-layer RBAC	Route middleware + Form Request authorize() + Service ownership check
Sensitive File Exposure	Nginx deny + .gitignore	.env, .git, composer files blocked by Nginx; .env gitignored
Clickjacking	HTTP header	X-Frame-Options: SAMEORIGIN via Nginx

SECTION 10

Deployment & Infrastructure

SERVICE	IMAGE	ROLE	PORT
pm_nginx	nginx:1.25-alpine	HTTP server; proxies PHP to pm_app via FastCGI	8080→80
pm_app	PHP 8.2-FPM	Laravel runtime; OTP generation, mail dispatch	9000 (internal)
pm_db	mysql:8.0	Persistent database with healthcheck	3306 (internal)
pm_mailhog	mailhog/mailhog	Catches OTP emails and notifications locally	8025 UI / 1025 SMTP

Demo Credentials

ROLE	LOGIN METHOD	EMAIL	PASSWORD / OTP
admin	Password	admin@example.com	Admin@1234
product_manager	Password	manager@example.com	Manager@1234
customer	OTP	any valid email	Check MailHog at :8025

SECTION 11

Challenges & Solutions

OTP Token — Hashed Storage

Problem: Storing a 6-digit OTP in plaintext means a DB breach exposes all pending tokens immediately.

Solution: bcrypt hash stored in DB; plaintext exists only in memory and in the email.

Hash::check() performs constant-time verification.

Session Cart Without a Database Table

Problem: A DB-backed cart needs its own table, user association before checkout, and abandoned-cart cleanup jobs.

Solution: PHP session array keyed by product ID. Zero schema, automatically expires with session. Trade-off: cart lost on session expiry — persisting to DB after login is a natural future enhancement.

Price Snapshot on Order

Problem: Storing only a product FK in order_items means future price changes retroactively alter historical order totals.

Solution: `unit_price` copied from `$product->price` into `order_items` at checkout time. Historical orders are immutable.

Named Route Conflict — Two Login Methods

Problem: Laravel's `auth` middleware redirects to the named route `login`. FruGo has two login paths — the name must resolve to one primary entry point.

Solution: Password login is named `login` (Laravel's default expectation). The page includes a prominent link to the OTP form. All middleware redirects land on the password page, which offers OTP as an alternative.

SECTION 12

Testing Strategy

TEST TYPE	SCOPE
Feature – OTP Auth	Request → token created; correct verify → logged in + token deleted; expired → rejected; wrong → rejected
Feature – Shop	Approved products visible; pending/rejected hidden from public listing
Feature – Cart	Add, update, remove; customer-only (admin blocked with 403)

TEST TYPE	SCOPE
Feature – Checkout	Order created with correct line items and snapshotted prices; cart cleared after
Feature – Approval	Manager creates product → pending; admin approves → approved + email sent
Unit – ProductService	canModify() for own/foreign/admin user combinations

SECTION 13

Future Enhancements

AREA	ENHANCEMENT	PRIORITY
Image Upload	Replace text path input with file upload — store to S3 with auto-thumbnail generation	High
Persistent Cart	Save cart to DB on login so it survives session expiry and device switches	High
OTP Rate Limiting	Limit OTP requests per email/IP to prevent email flood abuse	High
Payment Integration	Stripe or Razorpay — collect payment before confirming order	High
Admin Order Management	Admin view of all orders with status update controls	Medium
Automated Tests	PHPUnit feature tests for OTP, cart, checkout, and approval flow	Medium
CI/CD Pipeline	GitHub Actions: lint → test → Docker build → deploy on push to main	Medium

AREA	ENHANCEMENT	PRIORITY
CSP Header	Content-Security-Policy to prevent injected script execution	Low

SECTION 14

Conclusion

FruGo demonstrates a complete, production-structured Laravel application combining a customer-facing e-commerce shop with a staff back-office — differentiated entirely by role-based access control within a single codebase.

The passwordless OTP authentication provides frictionless customer onboarding while bcrypt-hashed token storage and immediate post-use deletion meet the same security bar as traditional password authentication. Price-snapshotted orders and the product approval workflow form a coherent, trust-safe buying experience that can scale to real production use with the enhancements identified above.

REPOSITORY

Source code, deployment scripts, and configuration: github.com/vasant24konar/FruGo